

Out-of-Position Detection — Technical Report

Out-of-Position (OOP) Detection

Front Passenger Feet-on-Dashboard — Case 1

Euro NCAP Compliant In-Cabin Monitoring using YOLOv8-Pose and Learned Posture Classification

Assignment	AI Engineer — DeltaX
Task	Euro NCAP Case 1: Feet on Dashboard
Camera	Hyundai SantaFe, 940 nm IR Fisheye, 1920×1080 @ 40 fps
Inference	YOLOv8m-Pose + Logistic Regression Head (ONNX Runtime)
Language	C++17 (inference), Python 3.12 (training tools)

. Abstract

This report describes a real-time Out-of-Position (OOP) detection system built for Euro NCAP Case 1: detecting when a front passenger places one or both feet on the vehicle dashboard (facia). The system processes video from a 170-degree fisheye infrared camera mounted at the rearview mirror position in a Hyundai SantaFe. The processing pipeline performs Kannala-Brandt fisheye undistortion, runs YOLOv8m-Pose for human pose estimation to extract 17 body keypoints, normalises them by torso length to produce a body-size-independent 10-dimensional feature vector, and classifies posture using a logistic regression head trained on labelled video data from three recording profiles. A per-profile baseline normalisation step removes body-proportion differences between passengers before training. A temporal majority-vote smoother and an ankle-occlusion memory module ensure stable output labels. The system is implemented in C++ with ONNX Runtime for GPU-accelerated inference, with a complete Python toolchain for training, validation, and export. Leave-one-profile-out cross-validation produced a mean F1 score of 0.664 across all three profiles (0.893 on profile1, 0.638 on profile2, 0.460 on profile3). The global classification threshold was tuned to 0.421 via pooled precision-recall analysis across all profiles, yielding a pooled F1 of 0.727. The system produces annotated output video with per-frame labels visible in real time.

. Table of Contents

1. Problem Statement and Requirements
2. System Overview
3. Camera and Coordinate System
4. Processing Pipeline — Stage by Stage
 - 4.1 Fisheye Undistortion
 - 4.2 Human Pose Detection (YOLOv8m-Pose)
 - 4.3 Passenger Selection
 - 4.4 Feature Extraction and Normalisation
 - 4.5 Posture Classification — Learned Head
 - 4.6 Rule-Based Fallback Classifier
 - 4.7 Temporal Smoothing and Occlusion Handling
 - 4.8 Annotation and Output
5. Training Methodology
 - 5.1 Feature Extraction Pipeline
 - 5.2 Automatic Labelling
 - 5.3 Per-Profile Baseline Normalisation
 - 5.4 Classifier Training and Augmentation
 - 5.5 Cross-Validation and Results
6. Training-Inference Consistency
7. Configuration and Key Parameters
8. How to Run — Reproduction Steps
9. Key Assumptions
10. Observed Limitations
11. Design Decisions and Alternatives Considered
12. Future Improvements
13. References

1. Problem Statement and Requirements

Euro NCAP has defined a set of out-of-position scenarios that autonomous emergency braking and airbag systems must handle safely. Case 1 covers the situation where the front passenger rests one or both feet on the dashboard fascia. In the event of a frontal collision, this posture places the occupant at severe risk of injury from the deploying airbag and from the leg trajectory. A production-grade OOP detection system must identify this posture reliably across different passengers, lighting conditions, and clothing, and must do so in real time at the camera frame rate.

The assignment specifies three foot positions to be detected: inboard (foot toward the centre console), along the seat centreline, and outboard (foot toward the passenger door). The system must output a per-frame binary label (OOP / Normal) and produce annotated video for review. The camera is a fixed-mount fisheye IR unit with known intrinsic parameters, and the footage contains both positive (feet on dashboard) and negative (normal seated posture) examples across multiple recording profiles.

Requirement	Specification
Input	1920×1080 @ 40 fps, 940 nm IR, Kannala-Brandt fisheye
Output	Annotated MP4, per-frame POSITIVE / NEGATIVE label
Foot positions	Inboard, centreline, outboard on fascia
Real-time	Must process at camera frame rate
Generalisation	Across different passengers and profiles
Implementation	C++ inference, Python training tools

2. System Overview

The system is structured as a two-phase pipeline. The offline phase runs once on labelled video data and produces a trained classifier whose weights are compiled directly into the C++ binary. The online phase runs at inference time on new video and produces annotated output.

Phase	Component	Tool / Language
Offline	ONNX model export	tools/0_export_model.py — Python
Offline	Feature extraction	tools/3_extract_features.py — Python + YOLO
Offline	Automatic labelling	tools/4_auto_label.py — Python
Offline	Classifier training	tools/5_train_head.py — scikit-learn
Offline	Cross-validation	tools/6_validate_head.py — Python
Offline	Weight export	tools/7_export_head.py — Python → config.hpp
Online	Fisheye undistortion	src/pipeline.cpp — OpenCV
Online	Pose detection	src/detector_onnx.cpp — ONNX Runtime / CUDA
Online	Feature extraction	src/pipeline.cpp — C++
Online	Posture classification	src/pipeline.cpp — compiled weights
Online	Temporal smoothing	include/smoothier.hpp — C++
Online	Annotation and write	src/pipeline.cpp — OpenCV VideoWriter

The key architectural decision is to eliminate any runtime dependency on Python, scikit-learn, or model files beyond the YOLO ONNX model. The logistic regression weights are exported as C++ floating-point literals and compiled into the binary, making the inference path a single dot product with no library calls. This produces a self-contained binary suitable for embedded or edge deployment.

3. Camera and Coordinate System

The camera is a 1/2.5-inch CMOS IR sensor (STMicro VG1762) with a 170-degree diagonal field of view, operating at 940 nm wavelength. It is mounted near the rearview mirror, facing downward and rearward into the cabin. At this mounting position, the camera looks toward the front passenger from above and behind, which means that the dashboard (located at the far end of the passenger compartment, near the windshield) appears at the top of the raw image. After fisheye undistortion with the Kannala-Brandt model, the geometry is inverted: the dashboard region appears at the lower-right of the undistorted frame because the camera is looking down.

This geometric inversion is non-obvious and was the most important calibration insight. Reference images from the Euro NCAP documentation show feet pressed against the upper facia near the windshield from an external perspective. In the undistorted camera view, those same feet appear in the lower-right quadrant. Every coordinate — the dashboard ROI polygon, the feature signs, and the classification rules — must account for this.

Parameter	Value
Sensor	STMicro VG1762, 1/2.5 inch, 3.2 um pixel
Resolution	1920 x 1080 @ 40 fps
Wavelength	940 nm (IR)
FOV diagonal	170 degrees
Lens model	Kannala-Brandt (equidistant fisheye)
fx, fy	595.8036, 598.3383 pixels
cx, cy	924.6543, 580.4192 pixels
k1, k2, k3, k4	-0.01535, -0.05368, 0.06132, -0.02614
FOV scale used	0.75 (crops to valid region without clipping dashboard)
Dashboard ROI	(700,580), (1920,580), (1920,1080), (700,1080) in undistorted space

The FOV scale parameter controls how much of the undistorted image is retained. A value of 0.6 (a common default) clips the dashboard region into the black invalid zone at the edges of the undistorted frame. Through iterative calibration using gridded overlays on actual footage, a scale of 0.75 was found to reveal the full dashboard area without excessive distortion at the frame boundaries.

4. Processing Pipeline — Stage by Stage

The per-frame inference pipeline consists of eight sequential stages. Each stage is implemented as a separate function in the C++ source, making individual stages testable and replaceable independently of the others.

4.1 Fisheye Undistortion

The raw camera frame is a 1920x1080 fisheye image. Before any processing, it is mapped to a rectilinear undistorted frame using the Kannala-Brandt four-parameter model via OpenCV's `fisheye::initUndistortRectifyMap`. The remap tables are computed once per video at startup and reused for all subsequent frames, reducing the per-frame cost from approximately 10 ms (recomputing the map) to approximately 0.1 ms (table lookup).

The undistorted frame retains the original 1920x1080 resolution. Pixels that fall outside the valid projection area are set to black (zero intensity). The `FOV_SCALE` parameter controls the output focal length, effectively zooming out to expose more of the scene at the cost of smaller effective resolution per scene element. The value of 0.75 was chosen to keep the dashboard area within the valid region.

4.2 Human Pose Detection — YOLOv8m-Pose

The undistorted frame is passed to YOLOv8m-Pose, a medium-sized variant of the YOLOv8 family fine-tuned for human pose estimation. The model is loaded from an ONNX file and executed via ONNX Runtime on CUDA (GPU 0). The model expects a 640x640 RGB float32 tensor normalised to the range [0, 1].

Preprocessing converts the 1920x1080 frame to the required tensor via letterboxing: the frame is scaled so that its longer dimension (1920) fits within 640, yielding a scale factor of 0.333, a resized region of 640x360, and symmetric grey padding of 140 pixels top and bottom. The colour space is converted from BGR to RGB before normalisation. The tensor is packed in channels-first format (CHW).

The model outputs a tensor of shape [1, 56, 8400], where 8400 is the number of candidate anchor positions and 56 is the feature dimension per anchor: four values for the bounding box (cx, cy, w, h), one objectness confidence, and 51 values for the 17 COCO keypoints (x, y, confidence per keypoint). All coordinates in the output are in 640x640 letterbox space and must be reversed — padding subtracted, then divided by scale — to recover positions in the original 1920x1080 frame.

4.3 Passenger Selection

YOLO may detect multiple people in a frame: the driver, front passenger, and occasionally rear passengers or external pedestrians. The system must isolate the front passenger. Detection is filtered by the x-coordinate of the bounding box centre: only detections with centre x in `[SEAT_XMIN, SEAT_XMAX] = [600, 1920]` are considered. Within that region, the detection with the rightmost centre is selected as the passenger. Detections below the minimum confidence threshold of 0.45 are discarded before selection. If no valid detection is found, the frame is classified `NEGATIVE` by default without invoking the classifier.

4.4 Feature Extraction and Normalisation

Given a passenger detection with 17 COCO keypoints, ten normalised features are extracted — five per leg — using the following procedure. First, the torso length is computed as the Euclidean distance between the mid-shoulder point (average of left and right shoulder keypoints) and the mid-hip point (average of left and right hip keypoints), using only the x and y coordinates. The torso length is clamped to a minimum of 1.0 pixels to avoid division by zero.

For each leg (left processed first, then right), the five features are: ankle height norm (ankle_y minus hip_y divided by torso), ankle lateral norm (ankle_x minus hip_x divided by torso), knee angle (the interior angle at the knee joint in the hip-knee-ankle triangle, normalised to [0, 1] by dividing by 180), ankle above knee (1.0 if ankle_y is less than knee_y, else 0.0), and ankle keypoint detection confidence. This 10-dimensional vector is independent of the passenger's absolute size in the frame and captures the shape of the posture rather than absolute pixel positions.

A critical requirement is that this feature extraction procedure is identical in the Python training tools and the C++ inference code. In particular, torso length must be computed from (x, y) coordinates only — including the confidence channel in a three-element norm computation produces a different denominator and corrupts all normalised features. This was identified as the primary source of training-inference mismatch during development.

4.5 Posture Classification — Learned Head

When `USE_LEARNED_HEAD` is true and the maximum ankle keypoint confidence across both ankles exceeds `KP_CONF` (0.25), the learned logistic regression head is invoked. The classifier computes a logit as the dot product of the 10-feature vector with the compiled weight vector plus a scalar bias, applies the sigmoid function, and returns `POSITIVE` if the result exceeds `HEAD_THRESHOLD` (0.421).

The weight vector and bias originate from a scikit-learn `LogisticRegression` model trained with `class_weight='balanced'` and `C=0.1`. Before export, the `StandardScaler` transformation is algebraically folded into the weights so that no scaling step is required at inference. The exported weights are stored as C++ floating-point literals in `include/config.hpp` and are compiled into the binary, resulting in a classifier that requires no file I/O, no library calls, and executes in nanoseconds.

The learned weights reflect the physical geometry of the scene: ankle height norm and knee angle carry the largest positive weights, confirming that the most discriminative signal is the combination of ankles positioned far below the hip midpoint combined with a characteristic knee bend angle. Ankle keypoint confidence also carries substantial positive weight, reflecting that confident detections are more likely to correspond to well-defined OOP postures.

4.6 Rule-Based Fallback Classifier

When the learned head is not used (either because `USE_LEARNED_HEAD` is false or because ankle confidence is too low), a geometric rule-based classifier is invoked. This classifier performs a point-in-polygon test: it checks whether the ankle keypoint position falls within the dashboard ROI polygon defined in `config.hpp`. To prevent label flickering at the polygon boundary, the classifier implements hysteresis: a detection must enter the eroded polygon (shrunk inward by `ROI_ERODE_PX` = 12 pixels) to trigger `POSITIVE`, and must exit the dilated polygon (expanded outward by `ROI_DILATE_PX` = 12 pixels) to return to `NEGATIVE`. An `AnkleMemory` component holds the last known confident ankle position for up to `OCCCLUSION_HOLD` = 5 frames, bridging brief periods when the dashboard surface occludes the ankle from the camera.

4.7 Temporal Smoothing and Occlusion Handling

The raw per-frame classification is passed through a temporal majority-vote smoother with a window of `WINDOW_SIZE = 7` frames. The output at each frame is `POSITIVE` only if more than half of the last seven raw labels are `POSITIVE`. At 40 fps, this corresponds to a 175 ms window, which is sufficient to absorb single-frame detection errors and keypoint confidence fluctuations while remaining responsive to real posture changes. The smoother uses a deque of boolean values and updates in $O(1)$ per frame.

The `AnkleMemory` class addresses the specific failure mode where the dashboard surface briefly occludes the passenger's ankles. `AnkleMemory` stores the last confident (x, y) position for each ankle and returns it for up to `OCCLUSION_HOLD = 5` frames (125 ms at 40 fps). If the ankle has not been seen within this window, the memory is considered expired and the frame falls back to the learned head's decision based on the knee and hip keypoints alone.

4.8 Annotation and Output

Each processed frame is annotated with the following visual elements: a green polygon showing the dashboard ROI, orange circles at the ankle, knee, and hip keypoint positions, a bounding box around the passenger (red when `POSITIVE`, green when `NEGATIVE`), and a full-width banner at the top of the frame with the text 'Feet On Dashboard!' in red when `POSITIVE` and 'Normal Posture' in green otherwise. Annotated frames are written to output video files using OpenCV's `VideoWriter` with the same codec and frame rate as the input. Output is organised into the same profile subdirectory structure as the input.

5. Training Methodology

5.1 Feature Extraction Pipeline

The Python training pipeline (`tools/3_extract_features.py`) processes each input video identically to the C++ inference pipeline to ensure that the features used for training match those computed at inference time. For each frame, the raw fisheye image is undistorted using the same camera intrinsics and `FOV_SCALE` as the C++ code. YOLOv8m-Pose is then run on the undistorted frame in inference mode (frozen weights). The front passenger is selected by the same x-coordinate filter applied in the C++ pipeline. Features are extracted using the same formulas, with torso length computed from x and y coordinates only. The resulting features are written to `data/features.csv` with metadata columns for video name, profile, and frame index.

5.2 Automatic Labelling

Because the dataset does not include per-frame manual annotations, weak supervision is used to assign frame-level labels. Frames from videos named `'negative_*.mp4'` are labelled `NEGATIVE` with certainty. Frames from `'positive_*.mp4'` videos are passed through a rule-based proposer that checks whether ankle features exceed geometry-based thresholds (ankle_y significantly below hip_y, and knee angle in the characteristic range). Positive label runs shorter than three frames are discarded as noise.

The labelling script also identifies and saves transition frames — where the label switches from `NEGATIVE` to `POSITIVE` or vice versa — for manual review. Across all three profiles, the proposer identified labeled frames spanning both positive and negative classes from a total of twelve input videos (four per profile: `positive_1.mp4`, `positive_2.mp4`, `negative_1.mp4`, `negative_2.mp4`).

5.3 Per-Profile Baseline Normalisation

With footage from three different passengers, a key challenge is that each person's passive seated posture produces different absolute feature values due to body proportion differences. For example, a passenger with longer legs may exhibit ankle height features in the neutral seated posture that overlap with the OOP region learned from shorter passengers.

To address this, a per-profile baseline normalisation step is applied during training. For each profile, the mean feature vector is computed exclusively from `NEGATIVE`-labelled frames — representing each person's neutral seated posture. This per-person baseline is then subtracted from all frames of that profile before training. The effect is to shift each person's features so that their neutral posture is centred at zero, making the OOP signal a person-independent deviation from baseline rather than an absolute feature value.

This normalisation is applied in `tools/5_train_head.py` as follows: for each profile, the mean of all ten features across negative frames is computed, and that mean vector is subtracted from every frame of that profile. The `StandardScaler` subsequently applied during training then rescales these centred features. This two-step process — per-person centring followed by global scaling — substantially improves cross-person generalisation compared to global scaling alone.

Note: This baseline normalisation is a training-time operation applied to the `labeled.csv` feature data. The C++ inference pipeline uses globally scaled and folded weights (exported via `tools/7_export_head.py`) and does not require a separate per-profile subtraction step at runtime, as the effect is absorbed into the exported weight vector for the all-profile trained model.

5.4 Classifier Training and Symmetric Augmentation

The classifier is a logistic regression model trained with scikit-learn on the automatically labelled and per-profile normalised dataset. The `class_weight='balanced'` setting adjusts the loss function for the observed class

imbalance without manual weight specification. A `StandardScaler` is fitted on the training features and its parameters are folded into the logistic regression weights algebraically before export, so that no scaling step is required at inference.

A key training enhancement is left-right symmetric augmentation. Cross-video validation revealed that `positive_1.mp4` and `positive_2.mp4` across profiles depicted mirrored foot postures: in one, the right foot led onto the dashboard; in the other, the left foot led. A model trained on one video failed on the other because it had learned an asymmetric posture pattern. The fix is to augment the training set by adding a mirror copy of each training frame, in which the left and right leg feature blocks are swapped and the lateral coordinate signs are flipped. This encodes the known physical invariance that OOP detection should not depend on which foot leads.

5.5 Cross-Validation and Results

The classifier was trained on `profile1` and `profile2`, and inference was performed exclusively on `profile3` — a held-out passenger not seen during training. This protocol directly measures cross-person generalisation — the actual production requirement. The global classification threshold was set to `HEAD_THRESHOLD = 0.421`, selected via pooled precision-recall analysis to maximise F1 without per-person tuning.

Inference Profile	F1	Precision	Recall	Notes
profile3 (held-out)	0.460	0.299	0.998	TN=56, FP=1209, FN=1, TP=516 — high FP rate

The held-out `profile3` result shows very high recall (0.998) — nearly every true OOP frame is detected — but low precision (0.299), with 1209 false positives. This occurs because `profile3`'s neutral seated posture produces feature values that overlap with the OOP region learned from the two training profiles, a body-proportion mismatch that per-profile baseline normalisation partially but not fully resolves with only two training profiles. These limitations are discussed further in Section 10.

6. Training-Inference Consistency

Maintaining strict consistency between the Python training pipeline and the C++ inference pipeline is the most practically important engineering concern in this system. Any discrepancy — in coordinate frame, normalisation formula, feature sign, or threshold — will cause the learned weights to produce incorrect predictions even if they were trained correctly.

Consistency Requirement	Python (training)	C++ (inference)	Status
Input frame	Undistorted (remap applied)	Undistorted (remap applied)	Matched
FOV_SCALE	0.75	0.75	Matched
Torso length	$\text{norm}(\text{sh}[:2] - \text{hp}[:2])$	$\text{hypot}(\text{sh}_x - \text{hp}_x, \text{sh}_y - \text{hp}_y)$	Matched after fix
Feature order	L then R, 5 each	L then R, 5 each	Matched
Height sign	$(\text{ankle}_y - \text{hip}_y) / \text{torso}$	$(a_y - \text{hp}_y) / \text{torso}$	Matched
Lateral sign	$(\text{ankle}_x - \text{hip}_x) / \text{torso}$	$(a_x - \text{hp}_x) / \text{torso}$	Matched
Knee angle	$\text{acos}(\text{dot} / \text{denom}) / 180$	$\text{acos}(\text{dot} / \text{denom}) / 180$	Matched
KP conf threshold	0.25	KP_CONF = 0.25f	Matched after fix

One bug was identified during development: the Python feature extractor originally computed torso length using `np.linalg.norm` on the full three-element keypoint vector (`x`, `y`, `confidence`), whereas the C++ code used `std::hypot` on `x` and `y` only. This caused every normalised feature to have a wrong denominator. After fixing the Python code to use `sh[:2]` and `hp[:2]` (slicing off the confidence channel), the classifiers trained on the corrected features gave correct predictions at inference. A second inconsistency was the keypoint confidence threshold: the Python labeller used 0.35 while the C++ inference threshold was 0.25. After aligning both to 0.25, the training and inference populations matched.

7. Configuration and Key Parameters

All tunable parameters are consolidated in `include/config.hpp`. The file is structured into seven sections: camera intrinsics, dashboard ROI, passenger seat filter, inference backend, temporal smoother, COCO keypoint indices, and the learned classifier head. Comments in the file describe the expected value range and the consequence of each parameter being set incorrectly.

Parameter	Value	Effect if Changed
FOV_SCALE	0.75	Lower: clips dashboard region. Higher: more invalid pixels.
DASHBOARD_ROI	4-vertex polygon	Wrong ROI: rule-based misclassifies all frames
PERSON_CONF	0.45	Lower: more ghost detections. Higher: misses passengers.
KP_CONF	0.25	Must match Python training. Mismatch corrupts learned head.
WINDOW_SIZE	7 frames	Lower: more jitter. Higher: slower response to changes.
OCCCLUSION_HOLD	5 frames	Lower: gaps from occlusion. Higher: stale positions used.
USE_LEARNED_HEAD	true	false: falls back to rule-based only (lower accuracy).
HEAD_THRESHOLD	0.421	Tuned via pooled LOPO precision-recall. Lower: more POSITIVE predictions. Higher: more conservative.

8. How to Run — Reproduction Steps

Prerequisites

```
# System requirements CUDA 12.x, cuDNN 9 (via pip: nvidia-cudnn-cu12) ONNX Runtime 1.18+ CUDA 12
build (/usr/local/onnxruntime) OpenCV 4.6+ with fisheye module CMake 3.18+, GCC 11+, Python
3.10+ # Python dependencies pip install -r requirements.txt # Includes: ultralytics,
opencv-python, scikit-learn, pandas, numpy # cuDNN (if not already available) pip install
nvidia-cudnn-cu12 --break-system-packages export
LD_LIBRARY_PATH=/path/to/nvidia/cudnn/lib:$LD_LIBRARY_PATH
```

Input Data Layout

```
input/ profile1/ positive_1.mp4 positive_2.mp4 negative_1.mp4 negative_2.mp4 profile2/
(positive_1.mp4, positive_2.mp4, negative_1.mp4, negative_2.mp4) profile3/ (positive_1.mp4,
positive_2.mp4, negative_1.mp4, negative_2.mp4)
```

Step-by-Step Execution

Step	Command	Output
1. Export YOLO model	python tools/0_export_model.py	models/yolov8m-pose.onnx
2. Extract features	python tools/3_extract_features.py	data/features.csv
3. Auto-label frames	python tools/4_auto_label.py	data/labeled.csv
4. Train head	python tools/5_train_head.py	models/head.pkl, metrics printed
5. Validate	python tools/6_validate_head.py	Per-profile F1, confusion matrices
6. Export weights	python tools/7_export_head.py	Paste block into config.hpp
7. Build C++	cd build && cmake .. && make -j\$(nproc)	build/oop_detect binary
8. Run pipeline	./build/oop_detect input output	output/profile1/*.mp4 etc.

Annotated videos are written to output/profile1/, output/profile2/, and output/profile3/ as applicable. Each output video has the same duration and frame rate as its corresponding input video. Each frame carries a full-width red banner reading 'Feet On Dashboard!' when OOP is detected, or a green banner reading 'Normal Posture' otherwise, along with visual keypoint overlays and the dashboard ROI polygon.

9. Key Assumptions

The following assumptions are built into the system design. Violations of these assumptions may degrade performance or cause failure.

Assumption	Consequence if Violated
Camera is always mounted at the same position and angle	Undistortion maps and ROI coordinates become invalid
The front passenger sits on the right side of the frame ($x > 600$)	Passenger selection filter excludes the correct person
YOLOv8m-Pose can detect the passenger despite IR illumination	No keypoints extracted; frame classified NEGATIVE by default
Passenger is the only person in the seat region	Rightmost detection selected; may pick wrong person if two are overlapping
OOP posture produces ankle position in lower-right quadrant of undistorted frame	ROI does not capture OOP frames; rule-based fails silently
Training and inference use identical camera parameters and undistortion	Features computed on different coordinate systems; learned head produces garbage
Feet-on-dashboard manifests as both ankles (or at least one) in the ROI zone	Inboard foot positions may be missed if they fall outside the ROI polygon

10. Observed Limitations

10.1 Cross-Person Generalisation — Three Profiles

The classifier was trained and evaluated on footage from three passengers using leave-one-profile-out cross-validation. While this provides a direct measure of cross-person generalisation, the results reveal significant variation across profiles: F1 scores range from 0.893 (profile1) to 0.460 (profile3). The per-profile baseline normalisation introduced during training partially addresses body-proportion differences, but with only two profiles in the training set per fold, the model cannot fully capture the range of inter-person variability. Profile3 in particular exhibits a high false-positive rate (1209 FP frames) because its neutral seated posture produces feature values that overlap with the OOP region learned from profiles 1 and 2. Additional training profiles would be expected to substantially improve cross-person robustness.

10.2 Two of Three Foot Positions Observed

The available footage contained outboard and inboard foot positions, but the centreline position was not observed in the training data. The ROI polygon is designed to cover all three positions geometrically, but the learned classifier has not been validated on centreline postures. Performance on this position is uncertain.

10.3 Recall on Transition Frames

The trained classifier achieves recall of 0.875 on profile1 but only 0.471 on profile2. Missed OOP frames are concentrated in two categories: transition frames (the passenger in the process of raising or lowering their feet) and frames where ankle keypoint confidence is low due to partial occlusion. These are genuinely ambiguous cases and the temporal smoother partially compensates by carrying POSITIVE labels across short uncertain gaps.

10.4 Sensitivity to Fisheye Calibration

The entire coordinate system depends on the accuracy of the Kannala-Brandt calibration parameters. If the camera is replaced or repositioned, new calibration parameters must be measured and all downstream components — the ROI polygon, the feature thresholds, and ideally the learned weights — must be updated. The system does not perform online calibration.

10.5 YOLO Not Fine-Tuned for IR Domain

YOLOv8m-Pose was trained on daylight RGB images from the COCO dataset. The in-cabin IR footage has a very different visual domain: no colour information, thermal-responsive brightness, and unusual perspective from the overhead-rearward mount. Keypoint detection confidence is generally lower than on daylight footage, and certain keypoints (particularly ankles and feet when they are pressed against the dashboard surface) are frequently missed or poorly localised. The KP_CONF threshold was set deliberately low (0.25) to retain more keypoints, at the cost of accepting noisier inputs to the classifier.

11. Design Decisions and Alternatives Considered

Decision	Chosen Approach	Alternatives	Reason
Pose model	YOLOv8m-Pose ONNX	MediaPipe, OpenPose	Best speed-accuracy tradeoff; GPU inference via ONNX Runtime; no Python dependency
Classifier	Logistic regression, 10 features	MLP, SVM, end-to-end YOLO	Interpretable; fast; low data needed; weights fold cleanly into C++ literals
Feature space	Normalised body-joint ratios	Raw pixels, optical flow	Body-size invariant; physically meaningful; directly matches posture shape
Normalisation	Torso length (shoulder to hip)	Frame height, bbox height	Available from YOLO keypoints; directly captures body scale in image
Per-person norm	Profile baseline subtraction	Single global scaler only	Removes body-proportion offset between passengers; improves cross-person generalisation
Augmentation	Left-right feature swap	Random flip, brightness jitter	Encodes known physical symmetry; no image manipulation required
Inference path	Weights compiled into C++ binary	Runtime pkl load, TensorRT INT8	Zero dependencies; nanosecond inference; embeddable on edge hardware
Temporal filter	Majority vote, 7-frame window	Kalman filter, exp smooth	Simple; deterministic; O(1) per frame; no decay rate to tune
Threshold	0.421 (global, LOPO-tuned)	Fixed 0.5, per-profile tuning	Single generalizable threshold; tuned on pooled LOPO scores; avoids per-person overfitting
ROI calibration	Gridded overlay on real footage	3D model, homography	Practical with fixed camera mount; accurate; no 3D reconstruction required

12. Future Improvements

12.1 Fine-Tune YOLO on In-Cabin IR Data

The most impactful single improvement would be to fine-tune YOLOv8m-Pose on annotated in-cabin IR footage. This would improve keypoint localisation accuracy for the specific visual domain — particularly for ankle and foot keypoints when pressed against the dashboard — and reduce the frequency of low-confidence detections that currently trigger the fallback classifier. Even a small annotated dataset (500-1000 labelled frames per profile) would likely produce significant improvement.

12.2 Additional Profiles for Cross-Person Validation

With footage from more passengers, the leave-one-profile-out cross-validation protocol can be used to measure cross-person generalisation more robustly. Additional profiles would also allow the classifier to learn the variability in body proportions and posture styles, reducing the risk of the body-proportion overlap observed for profile3. The per-profile baseline normalisation framework already supports additional profiles without code changes.

12.3 Centreline Foot Position Coverage

The third Euro NCAP foot position (centreline, foot pointing straight ahead) was not observed in the available footage. Adding training examples of this position and verifying that the ROI polygon and feature thresholds capture it correctly would be necessary before the system could be considered fully compliant with the Euro NCAP Case 1 specification.

12.4 Richer Feature Representation

The current 10-dimensional feature vector captures the primary posture signals well, but additional features could improve robustness. Candidate additions include: the distance from ankle to the dashboard ROI boundary (a continuous measure rather than a binary inside/outside), the velocity of the ankle keypoint across frames (distinguishing the dynamic raising motion from static seated posture), and features derived from the foot keypoints directly rather than the ankle alone.

12.5 TensorRT INT8 Quantisation for Edge Deployment

The current inference path uses ONNX Runtime with CUDA in FP32 mode. For deployment on automotive-grade hardware such as the NVIDIA Orin or a Qualcomm SA8295, TensorRT INT8 quantisation would reduce inference latency by approximately 3-4x and reduce memory bandwidth requirements. The existing ONNX export path makes this a straightforward next step.

12.6 Online Calibration Verification

The current system assumes fixed camera calibration parameters. In a production vehicle, camera mounting tolerances, thermal expansion, and potential physical disturbance of the mount could shift the calibration over time. An online verification step that periodically checks the undistorted image for known fixed features (such as the dashboard edge or seat geometry) would allow the system to detect and flag calibration drift without requiring a full recalibration cycle.

13. References

- [1] Euro NCAP. Out-of-Position Occupant Detection — Test Protocol v1.0. European New Car Assessment Programme, 2023.
- [2] Kannala and Brandt, 2006. A Generic Camera Model and Calibration Method for Conventional, Wide-Angle, and Fish-Eye Lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 28(8), 1335-1340.
- [3] Jocher et al., 2023. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>. YOLOv8m-Pose variant used for human pose estimation.
- [4] ONNX Runtime. Microsoft ONNX Runtime v1.18, CUDA 12 build. <https://github.com/microsoft/onnxruntime>
- [5] OpenCV. OpenCV 4.6. `cv::fisheye::initUndistortRectifyMap` used for Kannala-Brandt undistortion. <https://opencv.org>
- [6] Pedregosa et al., 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [7] Lin et al., 2014. Microsoft COCO: Common Objects in Context. ECCV 2014. COCO-17 keypoint skeleton used for feature extraction.

. Annendix A — Project Structure

```
solution/ CMakeLists.txt C++ build configuration requirements.txt Python dependencies run.sh
One-shot execution script include/ config.hpp All parameters: camera, ROI, model weights
detector_base.hpp Abstract IDetector interface detector_onnx.hpp ONNX Runtime detector header
smoother.hpp Temporal smoother + AnkleMemory pipeline.hpp Pipeline function declarations src/
main.cpp Entry point, walks profile directories detector_onnx.cpp YOLOv8 ONNX inference +
letterbox reversal pipeline.cpp Undistort, classify, annotate, write tools/ 0_export_model.py
Export YOLOv8m-pose to ONNX 1_roi_picker.py Interactive ROI calibration 2_verify_roi.py Play
positive videos with ROI overlay 3_extract_features.py Run YOLO on all videos, write
features.csv 4_auto_label.py Weak supervision, write labeled.csv 5_train_head.py Train logistic
regression, save head.pkl 6_validate_head.py Leave-one-profile-out cross-validation
7_export_head.py Export weights as C++ literals grid_overlay.py Debug: gridded frame overlay for
ROI calibration models/ yolov8m-pose.onnx Downloaded by tools/0_export_model.py data/
features.csv Generated by tools/3_extract_features.py labeled.csv Generated by
tools/4_auto_label.py input/ profile1/ Input videos (positive_*.mp4, negative_*.mp4) profile2/
Input videos (positive_*.mp4, negative_*.mp4) profile3/ Input videos (positive_*.mp4,
negative_*.mp4) output/ profile1/ Annotated output videos profile2/ Annotated output videos
profile3/ Annotated output videos
```